Práctica de Laboratorio Subastas y Proyecto Bot Telegram

Subasta distribuida concurrente y autenticación multifactor con Pyro4

Juan David Figueroa
Universidad Politécnica Salesiana
Sede Sur, Quito, Ecuador

Christian Humberto Carrión
Universidad Politécnica Salesiana
Sede Sur, Quito, Ecuador

Josué Samuel Vele
Universidad Politécnica Salesiana
Sede Sur, Quito, Ecuador

Asignatura: Sistemas Distribuidos | Docente: Ing. Ricardo Proaño | Quito, noviembre de 2026

***Resumen*—Este informe documenta dos proyectos de programación distribuida con Pyro4 desarrollados para la asignatura Sistemas Distribuidos: un sistema de subastas en tiempo real y un servicio de autenticación multifactor (MFA) que entrega códigos de un solo uso mediante un bot de Telegram. La subasta comparte una única instancia de objeto remoto entre todos los participantes, protege cada operación con un Lock, versiona el estado para descartar consultas antiguas e informa a los clientes mediante sondeo desde un hilo independiente. El servicio MFA verifica la contraseña con PBKDF2-HMAC-SHA256, genera códigos de seis dígitos con el módulo secrets, los envía por la API de Telegram, conserva solo su hash, limita su vigencia a 180 segundos, los asocia a la conexión que superó el primer factor y bloquea la cuenta durante 60 segundos tras tres fallos. Ambos sistemas se ejecutaron en equipos distintos de una red local con un Name Server compartido. Las capturas muestran el registro de participantes, las ofertas, el cierre de la subasta y los accesos y rechazos del MFA. Se discuten además las limitaciones encontradas y las mejoras necesarias para un entorno real.**

***Palabras clave*—Pyro4, subastas distribuidas, concurrencia, autenticación multifactor, Telegram Bot API, PBKDF2, Name Server, Python.**

I. INTRODUCCIÓN

Los sistemas distribuidos exigen coordinar procesos que comparten estado y se comunican por red, lo que introduce problemas de concurrencia, consistencia y seguridad. Este informe presenta la parte integradora de la práctica de laboratorio: el caso de estudio de subastas en tiempo real (Sección 4 de la guía) y el proyecto avanzado de autenticación multifactor con un bot de Telegram (Sección 6) [1]. Los cuatro niveles introductorios de RPC (calculadora, mensajería con Name Server, banco con estado y llamadas asíncronas) se documentan en un informe aparte [2].

Ambos proyectos se construyen sobre Pyro4 [3] y se encuentran en el repositorio del equipo [4], organizado con una carpeta por proyecto (Fig. 1 y Fig. 2). La Sección II describe el diseño, la implementación y las pruebas de la subasta; la Sección III hace lo propio con el servicio MFA; la Sección IV resume la ejecución en red local y la Sección V presenta las conclusiones.

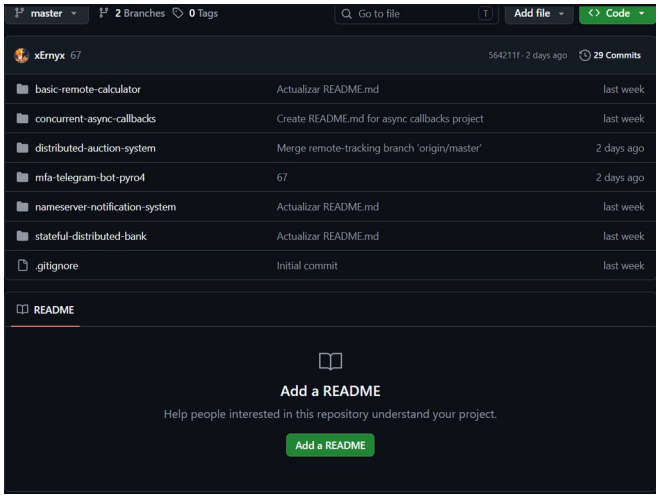


Fig. 1. Repositorio del equipo en GitHub con las carpetas de subastas y MFA ya publicadas.

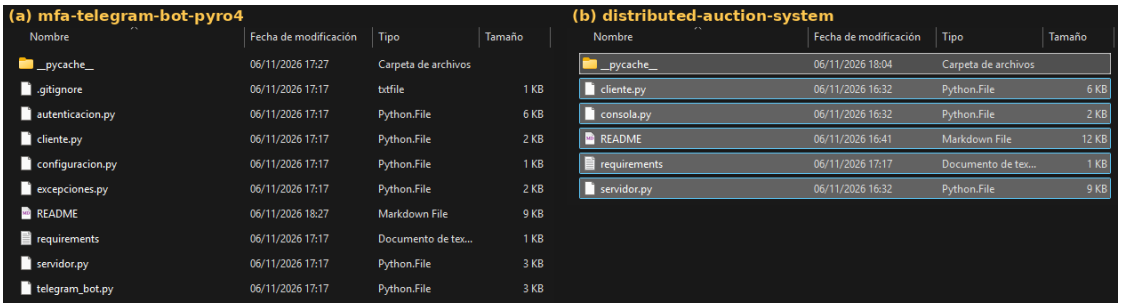


Fig. 2. Contenido de las carpetas de los dos proyectos en el equipo de desarrollo.

II. CASO DE ESTUDIO: SUBASTA DISTRIBUIDA

A. Requisitos y decisiones de diseño

La guía plantea una sala en la que varios clientes ofertan por un producto. El servicio debe registrarse en el Name Server, validar las ofertas, evitar condiciones de carrera, informar a los clientes de los cambios y reportar los errores con claridad [1]. La Tabla I resume cómo se resolvió cada requisito.

TABLA I
REQUISITOS Y DECISIONES DE DISEÑO DE LA SUBASTA

Requisito	Decisión de diseño	Evidencia
Registro en el Name Server	`register("subastas.central", uri)`	Código y Fig. 7
Estado compartido	Una sola instancia de `Subasta` para todos los clientes	Código y diagrama
Condiciones de carrera	`threading.Lock` en toda operación sobre el estado	Código
Validación de ofertas	Monto numérico, positivo, finito y mayor al precio; ligado al producto	Capturas de clientes
Notificación	Sondeo cada 0,25 s en un hilo propio, con versión del estado	Capturas de clientes
Errores claros	`ValueError` y `RuntimeError` con mensajes específicos	Capturas de clientes

B. Arquitectura y ciclo de vida

El sistema distingue dos roles (Fig. 3). El **operador** ejecuta `servidor.py`, prepara el producto (nombre, precio base y duración) e inicia la subasta desde su consola. Los **participantes** ejecutan `cliente.py`, se registran con un nombre, consultan la sala, ofertan y salen. Pyro4 expone los métodos públicos `registrar_cliente`, `consultar_estado`, `ofertar` y `salir_cliente`; los métodos `_preparar_subasta`, `_iniciar_subasta` y `_consultar_sala` empiezan con guion bajo, por lo que Pyro4 no los publica y solo el operador local puede invocarlos. Si un participante escribe `preparar` o `iniciar`, el cliente le informa que esa acción corresponde al operador.

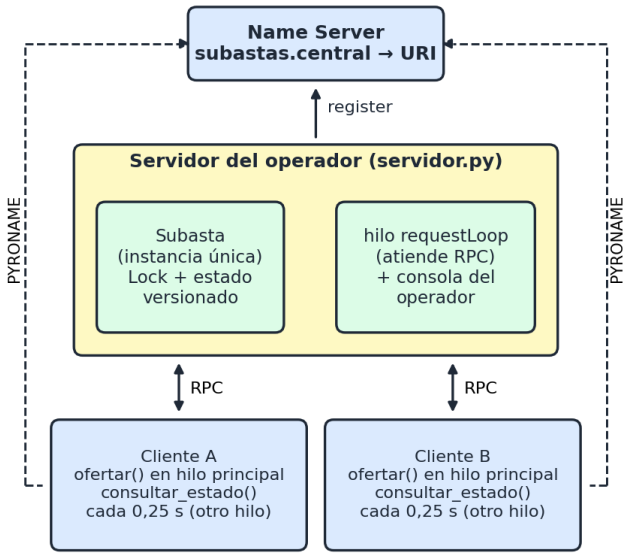


Fig. 3. Arquitectura de la subasta: una instancia compartida y un hilo de consulta por cliente.

La subasta atraviesa cuatro fases (Fig. 4). El paso a `finalizada` no usa un temporizador: `_obtener_estado` compara el instante de cierre con `monotonic()` cada vez que alguien consulta la sala y, al agotarse el tiempo, cambia la fase e incrementa la versión. Usar `monotonic()` evita que un ajuste del reloj del sistema altere el plazo.

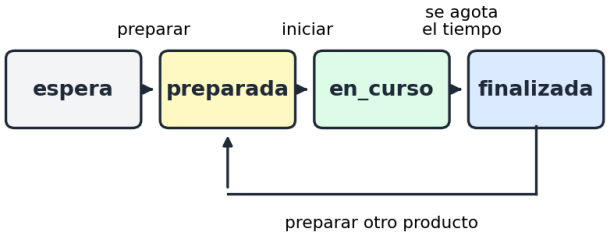


Fig. 4. Fases de la subasta y transiciones permitidas.

C. Concurrencia y consistencia

Se registra la **instancia** y no la clase (Fig. 5), de modo que Pyro4 la comparte entre todas las conexiones. Como el servidor atiende cada conexión en un hilo y el operador trabaja en paralelo, varias ofertas pueden llegar al mismo tiempo, y por eso el estado se protege con un `Lock` [5].

```
subasta = Subasta()
uri = servidor.register(subasta)  # instancia única
registro.register("subastas.central", uri)
hilo = Thread(target=servidor.requestLoop, daemon=True)
hilo.start()  # la consola del operador sigue activa
```

Fig. 5. Registro de la instancia compartida en `servidor.py`.

El método `ofertar` valida y actualiza dentro del mismo bloqueo (Fig. 6); si la validación estuviera fuera, dos clientes podrían superar el precio actual con el mismo valor y el segundo sobrescribiría al primero. Además de comprobar que el cliente esté registrado, el método exige que el monto sea `int` o `float` (se rechazan cadenas y booleanos), finito y positivo, y que supere el precio vigente.

```
def ofertar(self, nombre, monto, numero_subasta):
    monto = validar_numero_positivo(monto, "El monto")
    with self.bloqueo:  # validar y actualizar juntos
        self._validar_cliente(nombre)
        self._obtener_estado()
        if numero_subasta != self.numero_subasta:
            raise ValueError("El producto cambió...")
        if self.fase != "en_curso":
            raise RuntimeError("No hay una subasta...")
        if monto <= self.precio_actual:
            raise ValueError("La oferta debe superar...")
        self.precio_actual = monto
        self.mejor_postor = nombre
        self.version += 1
        return self._obtener_estado()
```

Fig. 6. Método `ofertar` de `servidor.py`, con los mensajes abreviados.

Dos mecanismos adicionales evitan decisiones tomadas con información vieja. El primero es `numero_subasta`: el cliente lo captura cuando el usuario empieza a escribir una oferta y lo envía junto con el monto, de modo que si el operador prepara otro producto mientras tanto, la oferta se rechaza con el mensaje "El producto cambió". El segundo es `version`, un contador que aumenta con cada cambio; el cliente descarta cualquier estado con una versión menor a la que ya mostró.

D. Notificación a los clientes mediante sondeo

Cada cliente lanza un hilo que llama a `consultar_estado` cada 0,25 s y muestra el estado solo si cambió y no fue consecuencia de su propia oferta. Los proxies de Pyro4 pertenecen al hilo que los creó (salvo que se use `_pyroClaimOwnership`), por eso ese hilo abre su propia conexión al mismo objeto con `Pyro4.Proxy(str(uri))`. La consola usa `prompt_toolkit` con `patch_stdout`, que permite imprimir avisos en vivo sin romper la línea que el usuario está escribiendo [6]. El operador recibe el mismo flujo de cambios a través de un hilo que consulta `_consultar_sala` y muestra cada nueva versión.

Se eligió el sondeo en lugar de callbacks porque estos obligan a que cada cliente publique su propio daemon accesible desde el servidor, algo que los cortafuegos y las redes con NAT dificultan. El costo es una latencia de hasta 0,25 s y un tráfico constante de cuatro consultas por segundo por cliente.

E. Resultados de la ejecución

La prueba se hizo con el servidor del operador y dos clientes, "Carrion" y "juan", en tres equipos conectados a la misma red local. Las capturas de la Fig. 7, la Fig. 8 y la Fig. 9 muestran la sesión completa, cuya secuencia se resume en la Tabla II. Antes de iniciar, el servidor rechazó `iniciar` sin producto preparado ("Primero prepara un producto para subastar") y los clientes vieron cómo la lista de participantes crecía a medida que se registraban.

TABLA II
SECUENCIA OBSERVADA EN LA SUBASTA

Momento	Evento	Resultado observado
Sala de espera	Carrion y juan se registran	Participantes: Carrion, juan
Sala de espera	`iniciar` sin producto	Rechazado por el servidor
Preparada	`preparar`: samsung s26, base $150, 150 s	Fase "Subasta preparada"
Preparada	Carrion ofrece 200 y 500; juan ofrece 150, 160 y 8000	"No hay una subasta en curso. Espera al inicio."
149,8 s	`iniciar`	En curso, $150.00, sin ofertas
≈145 s	Carrion ofrece 300	Aceptada; mejor postor: Carrion
142,1 s	juan ofrece 9000	Aceptada; mejor postor: juan
≈137 s	Carrion ofrece 1 000 000 000 000	Aceptada; mejor postor: Carrion
105,8 a 63,9 s	Consultas `estado`	Precio estable y tiempo decreciente
0 s	Se agota el tiempo	"Subasta finalizada"; ganador: Carrion
Final	juan escribe `salir`	"Saliste de la sala. Conexión cerrada."

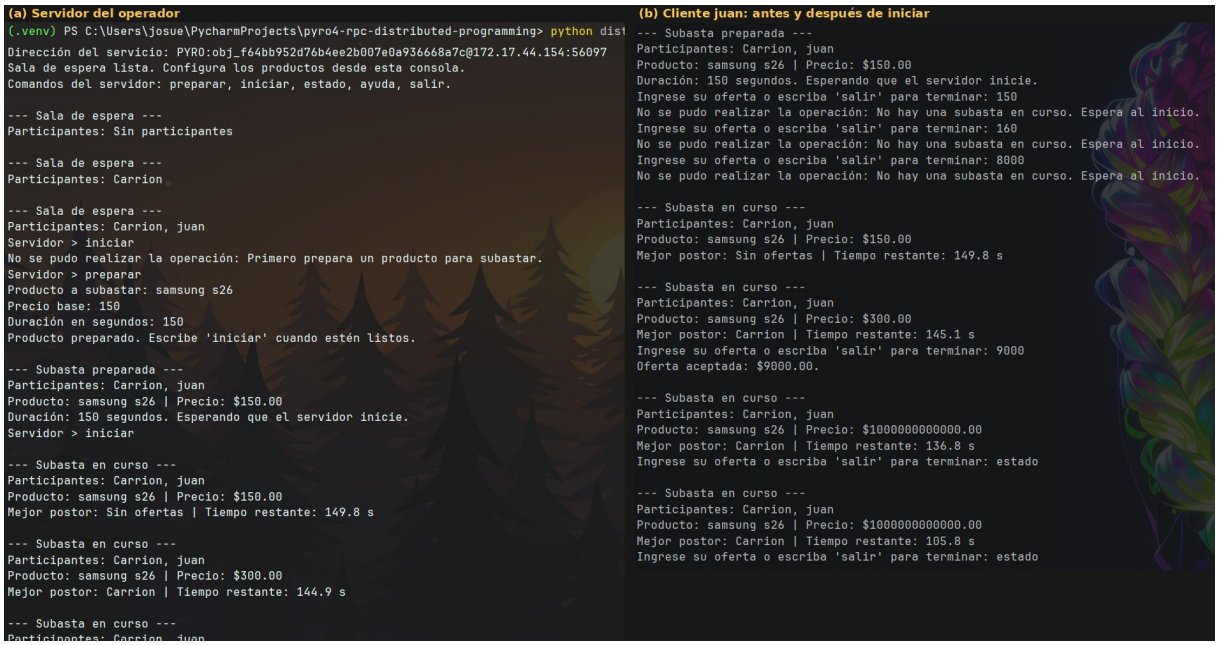


Fig. 7. Subasta: (a) consola del servidor con la preparación e inicio y (b) cliente juan antes y después de iniciar.

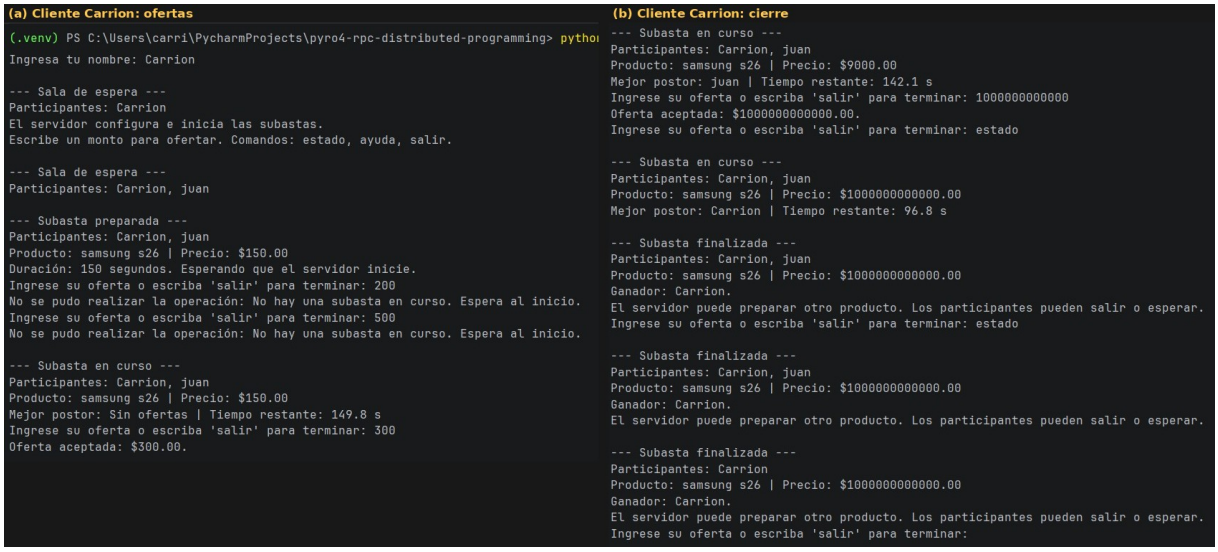


Fig. 8. Subasta: (a) ofertas del cliente Carrion y (b) finalización con el ganador.

Las ofertas anteriores al inicio se rechazaron con el mensaje esperado, y al arrancar la subasta ambos clientes recibieron el cambio de fase sin intervención. Las ofertas de 300, 9000 y 1 000 000 000 000 se aceptaron en orden y cada cliente vio la oferta del otro en su consola. Al agotarse el tiempo, ambos mostraron el mismo ganador. Cuando juan salió, la lista de participantes que ve Carrion se redujo a un solo nombre, lo que confirma que `salir_cliente` actualiza el estado compartido.

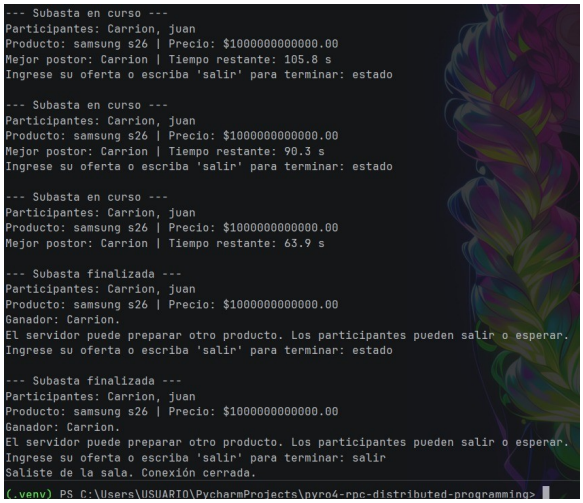


Fig. 9. Cliente juan: consultas de estado, cierre de la subasta y salida de la sala.

F. Alcance de las pruebas y limitaciones

- Las capturas no incluyen ofertas que no superen el precio, nombres repetidos, montos inválidos ni ofertas exactamente simultáneas. Esas validaciones están implementadas en el código y descritas en el README del proyecto, pero no se evidencian aquí.
- No existe un tope máximo: la oferta de un billón de dólares se aceptó porque cumple las reglas de "número positivo, finito y mayor al precio". Una regla de negocio (por ejemplo, un incremento máximo) evitaría errores de digitación.
- La identidad del participante es solo su nombre. `ofertar` comprueba que el nombre esté registrado, pero no que la conexión sea la que lo registró, de modo que un cliente malintencionado podría ofertar a nombre de otro.
- Todo el estado vive en memoria y se pierde al cerrar el servidor.

III. PROYECTO AVANZADO: MFA CON BOT DE TELEGRAM

A. Objetivo y requisitos

El proyecto implementa un servidor Pyro4 que autentica a un usuario en dos pasos: primero verifica usuario y contraseña y luego exige un código de seis dígitos que el servidor envía por Telegram. La guía pide que el código expire, que se limiten los intentos fallidos con un bloqueo temporal y que el envío se haga mediante la API de un bot [1]. Los códigos de un solo uso entregados por un canal independiente, junto con la limitación de intentos fallidos, son prácticas recomendadas por las guías de identidad digital del NIST [7].

B. Preparación del bot de Telegram

El bot se creó con BotFather mediante el comando `/newbot`, que entrega un nombre de usuario y un token de acceso a la API HTTP [8] (Fig. 10). El token otorga control total sobre el bot, por lo que se oculta en la captura. Cada persona que reciba códigos debe enviar `/start` al bot; de lo contrario Telegram no permite que el bot le escriba. Para saber a qué chat enviar los códigos, la utilidad `telegram_bot.py` consulta `getUpdates` y muestra únicamente los identificadores de chats privados, sin imprimir mensajes ni el token (Fig. 11a).

El servidor obtiene el token, en este orden, de la variable de entorno `TELEGRAM_BOT_TOKEN`, del archivo local `.telegram_token` (excluido de Git) o de una entrada oculta con `getpass`. Los errores HTTP no propagan la URL, que contiene el token.

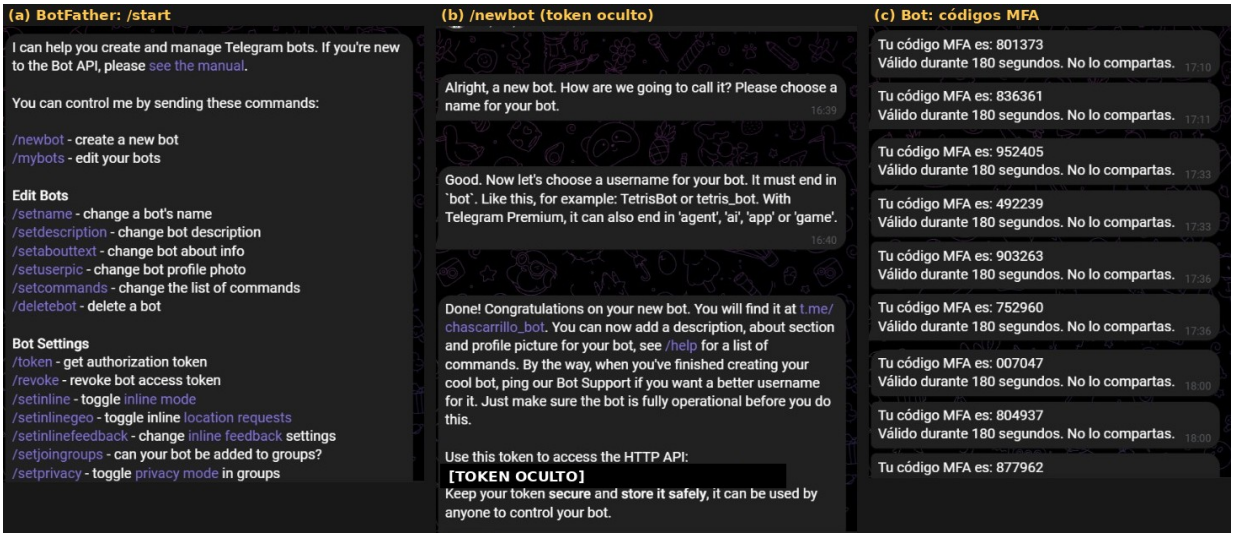


Fig. 10. Creación del bot en Telegram: (a) comandos de BotFather, (b) `/newbot` con el token oculto y (c) mensajes con los códigos MFA recibidos.

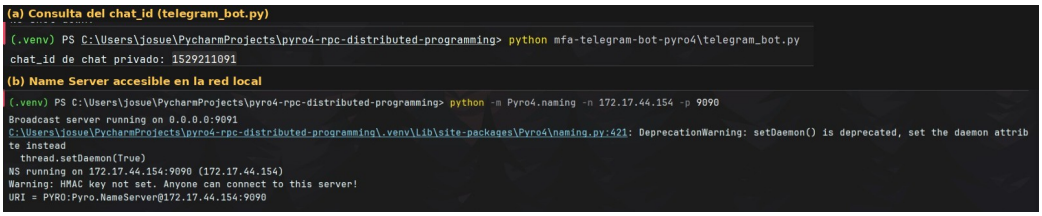


Fig. 11. Utilidades del proyecto MFA: (a) consulta del `chat_id` y (b) Name Server levantado en la red local.

C. Arquitectura y flujo de autenticación

El servicio se registra como `autenticacion.mfa` y expone dos métodos (Fig. 12). `iniciar_sesion(usuario, contraseña)` comprueba el primer factor, genera el código y lo envía; `verificar_codigo_mfa(usuario, código)` valida el segundo factor. El cliente mantiene abierto el mismo proxy durante todo el proceso, y el servidor guarda `Pyro4.current_context.client`, es decir, la conexión

que superó la contraseña. Al verificar, exige que la llamada llegue por esa misma conexión. Así, otro cliente que conozca el nombre de usuario no puede validar el código ni gastar sus intentos.

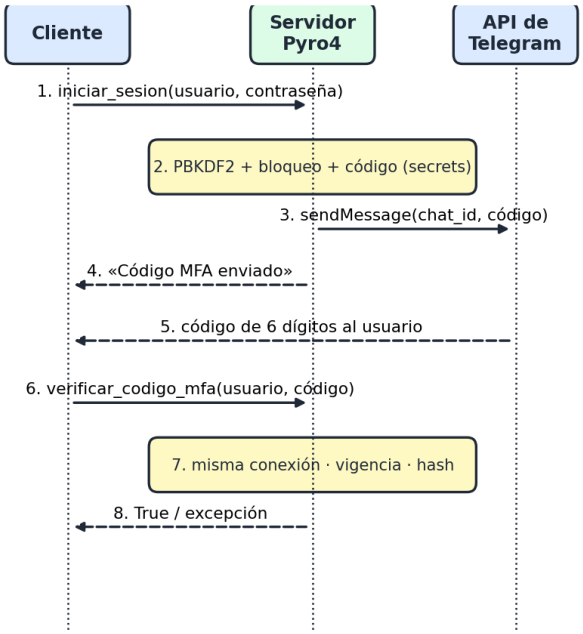


Fig. 12. Secuencia del flujo MFA entre cliente, servidor Pyro4 y la API de Telegram.

```
with estado.bloqueo:  # un bloqueo por usuario
    self._comprobar_bloqueo(estado)
    if not cuenta.comprobar_password(password):
        self._registrar_fallo(estado,
            CredencialesInvalidas("..."))
    codigo = f"{secrets.randbelow(1_000_000):06d}"
    self._telegram.enviar_codigo(
        cuenta.chat_id, codigo, self._validez)
    estado.resumen_codigo = sha256(
        codigo.encode("ascii")).digest()
    estado.expira_en = self._reloj() + self._validez
    estado.conexion = Pyro4.current_context.client
```

Fig. 13. Núcleo de `iniciar_sesion`: el código se activa solo si Telegram confirma el envío.

```
with estado.bloqueo:
    self._comprobar_bloqueo(estado)
    if (estado.resumen_codigo is None or estado.conexion
            is not Pyro4.current_context.client):
        raise CodigoIncorrecto("No hay un código...")
    if self._reloj() >= estado.expira_en:
        self._invalidar_codigo(estado)
        raise CodigoExpirado("El código expiró...")
    if not secrets.compare_digest(
            sha256(codigo.encode("ascii")).digest(),
            estado.resumen_codigo):
        self._registrar_fallo(estado, CodigoIncorrecto(
            "Código MFA incorrecto."))
    self._invalidar_codigo(estado)  # un solo uso
    estado.intentos = 0
    return True
```

Fig. 14. Núcleo de `verificar_codigo_mfa`, sin las validaciones de formato.

D. Medidas de seguridad implementadas

La Tabla III relaciona cada riesgo con el mecanismo que lo mitiga. La contraseña se almacena con PBKDF2-HMAC-SHA256, una sal aleatoria de 16 bytes y 600 000 iteraciones, el mínimo que recomienda OWASP para ese algoritmo [9], [10]. Los códigos se generan con `secrets`, el módulo de Python pensado para valores criptográficamente seguros [11], y del código solo se conserva su hash SHA-256.

TABLA III
RIESGOS Y MECANISMOS DE MITIGACIÓN DEL SERVICIO MFA

Riesgo	Mecanismo	Detalle
Robo de la base de contraseñas	PBKDF2-HMAC-SHA256	Sal de 16 bytes, 600 000 iteraciones y comparación con `compare_digest`
Adivinar el código	Código aleatorio de 6 dígitos y 3 intentos	3 intentos entre 1 000 000 de valores (0,0003 %)
Reutilizar un código	Un solo uso	Se consume al validar; un nuevo envío reemplaza al anterior
Código viejo	Vigencia de 180 s	Medida con `monotonic()`; lanza `CodigoExpirado`
Fuerza bruta	Bloqueo de 60 s	Suma fallos de contraseña y de código; `CuentaBloqueada`
Envío no confirmado	Activación condicionada	Solo se activa si Telegram responde `ok`; si no, `ErrorTelegram`
Código usado desde otra conexión	Vinculación a la conexión	Se compara con `Pyro4.current_context.client`
Enumerar usuarios	Mensaje único	“Usuario o contraseña inválidos” para ambos casos
Filtración del token	Configuración externa	Variable de entorno, archivo ignorado y errores sin URL

Las excepciones `CredencialesInvalidas`, `CodigoExpirado`, `CodigoIncorrecto`, `CuentaBloqueada` y `ErrorTelegram` heredan de `ErrorAutenticacion` y se registran con `register_dict_to_class`, la misma técnica usada para `SaldoInsuficiente` en el banco distribuido [2], de modo que el cliente las reciba con su tipo original y capture la clase base.

E. Resultados de la ejecución

El servidor registró dos cuentas, “cris” y “juan”, con sus contraseñas ocultas y el mismo `chat_id`, y quedó publicado como `autenticacion.mfa` (Fig. 15). Ambas cuentas compartieron el mismo chat durante la prueba, por lo que todos los códigos llegaron a una sola conversación de Telegram (Fig. 10c).

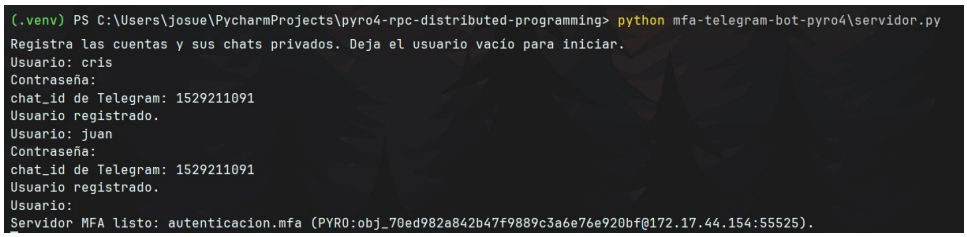


Fig. 15. Servidor MFA: registro de las cuentas y publicación de `autenticacion.mfa` en el Name Server.

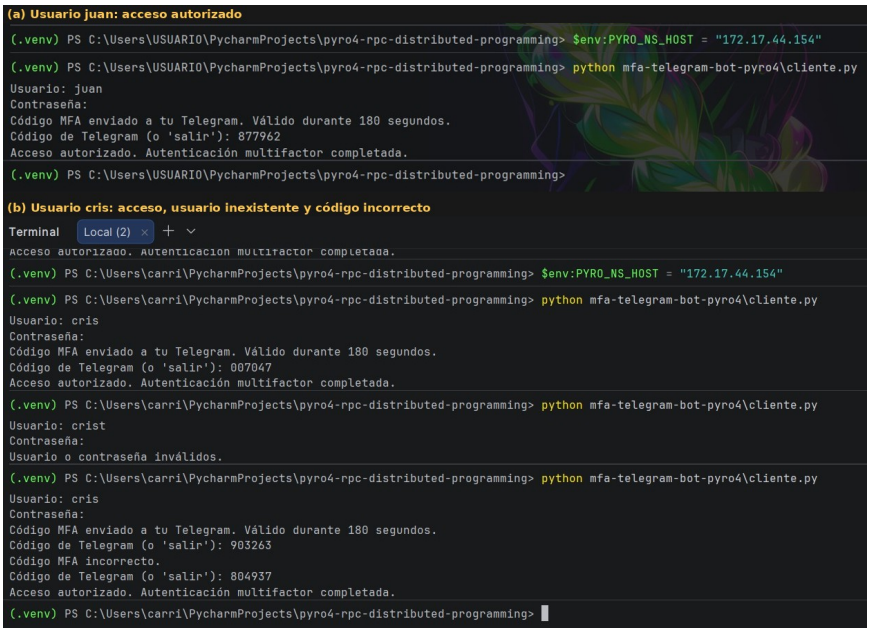


Fig. 16. Clientes MFA: acceso autorizado, usuario inexistente y código incorrecto seguido de uno correcto.

En la Fig. 16a, el usuario “juan” ingresó el código 877962, que coincide con el último mensaje del chat, y recibió “Acceso autorizado”. En la Fig. 16b, “cris” accedió con el código 007047. Al intentar con el usuario “crist”, que no existe, el servidor respondió “Usuario o contraseña inválidos”, el mismo mensaje que para una contraseña errónea. En la tercera ejecución de “cris”, el primer código ingresado (903263) fue rechazado con “Código MFA incorrecto”. Ese número corresponde a un mensaje de las 17:36 en el historial del chat, es decir, a un código de una sesión anterior que dejó de ser válido al emitirse uno nuevo. A continuación se ingresó el código vigente (804937, enviado a las 18:00) y el acceso se autorizó.

F. Alcance de las pruebas y limitaciones

- No se capturó el bloqueo tras tres fallos ni la expiración a los 180 s. Ambos comportamientos están implementados en `_registrar_fallo`, `_comprobar_bloqueo` y `verificar_codigo_mfa`, y descritos en el README, pero no se evidencian aquí.
- Pyro4 no cifra el tráfico por defecto: la contraseña y el código viajan en claro por la red local. Para redes no confiables se necesita TLS en Pyro4, además de HTTPS, que ya usa Telegram.
- Las cuentas se registran por consola al iniciar el servidor y se pierden al reiniciarlo, y la contraseña pasa por esa consola.
- PBKDF2 cumple las recomendaciones actuales con 600 000 iteraciones; para sistemas nuevos sin requisito de certificación, OWASP prefiere Argon2id [9].
- Compartir un mismo chat entre dos cuentas anula el sentido del segundo factor; en un entorno real cada usuario debe tener su propio chat.

IV. EJECUCIÓN EN RED LOCAL

Ambos proyectos se probaron en tres equipos. El Name Server se inició en una dirección de la red con `python -m Pyro4.naming -n 172.17.44.154 -p 9090` (Fig. 11b), y en los demás equipos se definió `PYRO_NS_HOST` para que Pyro4 lo localizara. Los servidores y clientes de ambos proyectos leen `Pyro4.config.NS_HOST` y `NS_PORT` al conectarse. La captura incluye la advertencia “HMAC key not set. Anyone can connect to this server!”: sin clave, cualquier equipo de la red puede consultar el Name Server y registrar nombres en él, lo que permitiría suplantar un servicio como `subastas.central` o `autenticacion.mfa`.

V. CONCLUSIONES

Los dos proyectos muestran cómo la política de instancia determina el diseño. En el banco del informe anterior, `session` daba a cada cliente su propio estado [2]. En la subasta, el estado debe ser único y compartido, y por eso se registra la instancia y se protege con un `Lock`. En el MFA, el servicio mantiene un

estado por usuario con su propio bloqueo, lo que permite atender cuentas distintas sin esperas cruzadas. La Tabla IV resume el cumplimiento de los requisitos de la guía.

TABLA IV
CUMPLIMIENTO DE LOS REQUISITOS DE LA GUÍA

Requisito	Cumplimiento	Evidencia
Subasta: registro y validación de ofertas	Cumple	Capturas y código
Subasta: concurrencia segura	Cumple	Código (`Lock`)
Subasta: notificación a clientes	Cumple, por sondeo	Capturas de clientes
MFA: código de 6 dígitos por Telegram	Cumple	Capturas de bot y clientes
MFA: expiración y bloqueo temporal	Implementado	Código; sin captura

El sondeo resultó suficiente para una sala pequeña, pero un sistema con muchos participantes necesitaría notificación por callbacks o un mecanismo de empuje para reducir tráfico y latencia. En seguridad, el principal hallazgo es que las defensas del servicio MFA (hashes, vigencia, bloqueo y vinculación a la conexión) se debilitan si el transporte no se cifra y el Name Server no se autentica. Como trabajo futuro se proponen TLS en Pyro4, un tope de oferta y un token de sesión para identificar a cada participante, persistencia de los usuarios, pruebas automatizadas de los casos de error no evidenciados, y la migración a Pyro5, sucesor de Pyro4.

REFERENCIAS

[1] “Práctica guiada: Programación distribuida con Pyro4 en Python,” guía de laboratorio, Sistemas Distribuidos, Univ. Politécnica Salesiana, Sede Sur, Quito, Ecuador, 2026.

[2] J. D. Figueroa, C. H. Carrión, and J. S. Vele, “Taller grupal práctica de laboratorio RPC con Python,” informe, Sistemas Distribuidos, Univ. Politécnica Salesiana, Sede Sur, Quito, Ecuador, 2026.

[3] I. de Jong, “Pyro – Python Remote Objects, release 4.82,” Pyro4 documentation. [Online]. Available: https://pyro4.readthedocs.io/. Accessed: Oct. 31, 2026.

[4] J. S. Vele, C. H. Carrión, and J. D. Figueroa, “pyro4-rpc-distributed-programming,” GitHub repository, 2026. [Online]. Available: https://github.com/JosueVele77/pyro4-rpc-distributed-programming

[5] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2007.

[6] J. Bernard, “prompt_toolkit documentation.” [Online]. Available: https://python-prompt-toolkit.readthedocs.io/

[7] P. A. Grassi et al., “Digital identity guidelines: Authentication and lifecycle management,” NIST, Gaithersburg, MD, USA, Spec. Pub. 800-63B, Jun. 2017.

[8] Telegram, “Telegram Bot API.” [Online]. Available: https://core.telegram.org/bots/api

[9] OWASP Foundation, “Password storage cheat sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[10] K. Moriarty, Ed., B. Kaliski, and A. Rusch, “PKCS #5: Password-based cryptography specification version 2.1,” IETF, RFC 8018, Jan. 2017.

[11] Python Software Foundation, “secrets — Generate secure random numbers for managing secrets,” Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/secrets.html